Retiring decryption authority, and the post-quantum path

Revocable access for oracle-based confidential execution. A draft ERC with a tested local reference implementation (Part A), and the full path from an ECDSA key to a post-quantum signature (Part B). Semih Civelek (Himess), 2026-10-22. Draft, not published.

Part A: retiring decryption authority

The problem.

- **History is decided again at every request.** In FHE and TEE systems on Ethereum, data sits on-chain as ciphertext handles, and a decryption oracle releases plaintext to whoever the host chain's access registry says may ask.
- **That authority is an ECDSA key, for life.** No system we examined lets an account end it. A leaked key, or a future quantum computer that derives it from a public key, inherits the account's whole history.
- **It is how the deployed system behaves today.**
  - On Zama fhEVM, one `allow` (140,780 gas on Sepolia) re-shares an account's data.
  - Delegating the account to a new wallet does not stop the old key.
  - 97.9% of sampled mainnet holders are ECDSA-rooted.

The idea.

- **Retirement.** `retireAccount()` makes every authorization view the oracle already reads return false for the account, in every role. There is no reverse transition.
- **Enforced in the views.** A threshold oracle of 13 nodes releases on any 5 responses. A rule in node software binds only once 9 operators run it, a cliff we measured; a rule in the views binds every verifier from the upgrade block.
- **The one-key rule.** Owner and thief hold the same key, so what the key alone can trigger must only remove authority. Hence:
  - a **successor** must be designated, accepted and matured before retirement;
  - a relayable **certificate** can trigger only that planned handover.

Evidence (local, three runs per check)	Result
Core on Zama fhEVM, both current lines	All registry suites pass; `retireAccount()` 50,099 gas (Osaka), 132,519 (Amsterdam pricing)
Unmodified verifier	Refuses a retired account on every request path, also when view calls fail. Stored results and final denials (TC15) need the hygiene profile: 35 cells conform, 8 violate, all unmodified
Second vendor; conformance kit	Ported to Fhenix CoFHE unchanged in mechanism (69 tests, 17 of 17 gate mutations caught). One vendor-neutral suite passes on both and fails when any of six gates is removed
Host-chain key disable (EIP-7851, EIP-8298)	Does not reach off-chain verifiers: in simulations of both, an unmodified verifier accepted the old key. A code-first verifier profile refuses it

Part B: the post-quantum path, end to end

The target. With the old ECDSA key assumed stolen:

1. no verifier releases anything on that key;
2. that key creates no right on chain;
3. the holder still decrypts past and future data with a post-quantum signature;
4. no upgrade undoes this without post-quantum authorization.

Three paths, each attacked with the old key (the real kms-worker plus local chains; three runs, every row as designed):

Path	Result
B1 carry the history to an ML-DSA-44 wallet, then retire (one transaction, 156,448 gas)	All four properties hold. Later grants to the old address are inert: apps grant to the wallet
B2 name the wallet as successor (7-day handshake), then retire or relay a certificate	All four hold. The wallet reads old and later handles through ERC-1271
B3 stay in place: EIP-7702 to the wallet, then EIP-7851 or EIP-8298	Off-chain holds for the old key's own signatures (its earlier delegations stand until revoked). On chain the old key still grants, delegates and can retire the account until a host chain enforces those drafts, so B3 does not combine with retirement

Who must do what.

- **Zama:**
  - Core in the registry; it works on the live line `v0.13.8` with no other change.
  - Code-first verification with an ML-DSA-sized ERC-1271 budget on `main`. The live line checks ECDSA only.
  - Optionally, native post-quantum signers.
- **Operators:** a scheduled post-quantum-only mode, which protects only with **9 of 13** enforcing. At 8 an ECDSA request was still released (5 workers forwarded it, enough for t + 1); at 9 it was withheld.
- **Governance:** operator-ratified upgrades. 9 of 13 ML-DSA-44 signatures passed the upgrade guard, 8 reverted, and a release without Core reverted.
- **Wallets:** ML-DSA-44 accounts with a relayable `execute`.
- **Ethereum:** EIP-7851 or EIP-8298, only for accounts that keep their address.

Costs.

- **Per request:** an ML-DSA-44 ERC-1271 check costs the oracle's node about 1.23M gas, five times today's cap; a native declared signer about 245 µs.
- **Per account:** the on-chain key costs 4.43M gas once (Osaka).
- **With EIP-8355 (Draft):** about 15,000 gas per check [estimate].
- **Outside the path, still classical:** the KMS's and coprocessors' signatures, the input proofs and governance.

What it is not: three limits

1. **Pre-emptive, not a clawback.** It protects accounts that retire or migrate before their key is used against them. Data a key already shared, published or had decrypted stays exposed.
2. **Oracle access only.** The key can still move funds and hold application roles, unless applications check retirement too.
3. **As final as the chain's governance.** Whoever can upgrade the registry can undo it. Today that is ECDSA multisigs with no timelock.

Status

- **The standard.** A registry-side ERC draft (Core, successor, certificate; clean under `eipw` except two placeholders) and a companion with candidate profiles.
- **The implementation.** A local prototype: not deployed, not audited. The ML-DSA libraries have no completed audit for this use: RustCrypto `ml-dsa`, `fips204`, `libcrux-ml-dsa`, PQClean (archived upstream), and the on-chain `evm-ml-dsa-verifier`. So the design requires two libraries to agree, or no library on t+1 nodes.
- **Reproduce.** The public repository's README gives one command per suite, including the post-quantum drill.